

Name and Surname: Matthew Magadza

Student ITS No: 402310062

Qualification: B S C I T Year of Study: 2025 Semester: 2nd

Assignment due date: _____ Date submitted: _____

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents Introduction

Main Body (with relevant subheadings) Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____

Date: _____

Contents

Question 1.....	3
A	3
B	4
C	5
Question 2.....	5
A	5
B	7
C	8
Question 3.....	9
A	9
B	9
C	12
Question 4.....	12
A	12
B	13
References	15

Question 1

A

C: > Users > user > OneDrive - Richfield Graduate Institute of Technology > Documents > 2nd year semster 2 > Programming assignment > question_1.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  //setting up the node
5  struct Node
6  {
7      int id;
8      Node* next;
9  };
10 // this function returns a node
11 Node* buildCircle(int N){
12     if (N <= 0) return nullptr;
13     Node* head = new Node(1,nullptr);
14     Node* prev = head;
15     for (int i= 2;i <= N; i++){
16         Node* temp = new Node(i, nullptr);
17         prev->next =temp;
18         prev = temp;
19     }
20     prev->next = head;
21     return head;
22 }
```

```
23
24 int josephus(int N,int M){
25     if (N ==0) return -1;
26     Node* curr = buildCircle(N);
27     Node* prev = nullptr;
28
29     while (curr->next != curr)
30     {
31         for (int i =0; i< M;i++){
32             prev = curr;
33             curr = curr->next;
34         }
35         prev->next = curr->next;
36         Node* toDelete = curr;
37         curr = curr->next;
```

C: > Users > user > OneDrive - Richfield Graduate Institute of Technology > Documents > 2nd year semster 2 > Programming assignment > question_1.cpp > ...

```
23
24 int josephus(int N,int M){
25     if (N ==0) return -1;
26     Node* curr = buildCircle(N);
27     Node* prev = nullptr;
28
29     while (curr->next != curr)
30     {
31         for (int i =0; i< M;i++){
32             prev = curr;
33             curr = curr->next;
34         }
35         prev->next = curr->next;
36         Node* toDelete = curr;
37         curr = curr->next;
38         delete toDelete;
39     }
40     int winner = curr->id;
41     delete curr;
42     return winner;
43
44 }
45
46 int main() {
47     int N, M;
48     cout << "Enter N (number of people): ";
49     cin >> N;
50     cout << "Enter M (number of passes): ";
51     cin >> M;
52
53     int winner = josephus(N, M);
54     cout << "The winner is person " << winner << endl;
55
56     return 0;
57 }
```

```
"C:\Users\user\OneDrive - Ric  X + v
Enter N (number of people):
```

```
"C:\Users\user\OneDrive - Ric  X + v
Enter N (number of people): 5
Enter M (number of passes): 1
The winner is person 3

Process returned 0 (0x0)   execution time : 8.511 s
Press any key to continue.
```

B

The time it takes for this implementation to run depends on two main factors: the number of people (N) involved and the number of passes (M) made during each elimination.

To break it down:

- Building the circle takes $O(N)$ time.
- Each elimination requires $O(M)$ for the passes, plus an additional $O(1)$ for the deletion process.
- Since there are $N - 1$ eliminations in total, we can calculate the overall time complexity as follows: $O(N) + (N-1) * O(M) = O(NM)$.

This aligns with the standard time complexity outlined by Cormen and colleagues in 2009, as well as Knuth in 1998. (Cormen, et al., 2009) (Knuth, 1998)

C

We only perform a single pass before every removal. Every step in $O(1)$ time is done $N-1$ times, giving an overall time complexity of $O(N)$. Theoretically, the time complexity is linear, but practically, the performance does suffer for large N due to memory management overhead—several new and delete calls can severely make things slow down (Cormen, et al., 2009). Additionally, cache inefficiency occurs because nodes in a linked list are distributed throughout memory, and this can lead to poor spatial locality (Knuth, 1998). We also have to consider fragmentation and pointer chasing, since all these operations involve indirect memory access, which can slow speed down when N is large (CP-Algorithms, 2024). Therefore, for $M = 1$, the time complexity is $O(N)$, but keep in mind that the actual time may be slightly more because of memory- and hardware-dependent matters.

Question 2

A

```
1  #include <iostream>
2  using namespace std;
3  /* this class is to create a self adjusting array which
4  works a bit like a queue all the operations start at the front
5  since this is not a normal array we have to set it to 100 to make it simple to adjust it
6  without constantly changing the actual length of the array */
7  class self_Adjusting_Arr{
8  private:
9      int arr[100];
10     int size;
11 public:
12     self_Adjusting_Arr(){
13         size = 0;
14     }
15     // adding to the front of the array
16     void insert_front(int val){
17         // shift the elements to the right
18         for (int i = size; i > 0; --i){
19             arr[i] = arr[i-1];
20         }
21         arr[0] = val;
22         size++;
23     }
24
25     /* this function or method allow you to tell if
26     the number we are looking for is in the array in the first place */
27     bool find(int val){
28         int index = -1;
29         for (int i = 0; i < size; i++){
30             if(arr[i] == val){
31                 index = i;
32                 break;
33             }
34         }
35         if (index == -1){
36             return false; //just in case it is not found
37         }
38     }
39 }
```

```
question_2a.cpp x
C:\Users\user> user > OneDrive - Richfield Graduate Institute of Technology > Documents > 2nd year semester 2 > Programming assignment > question_2a.cpp > self_Adjusting_Arr > self_Adjusting_Arr()
7  class self_Adjusting_Arr{
27  bool find(int val){
35      if (index == -1){
38          int temp = arr[index];
39          for (int j = index; j>0; --j){
40              arr[j] = arr[j -1];
41          }
42          arr[0] = temp;
43          return true;
44      }
45
46      void display(){
47          for (int i =0;i<size;i++){
48              cout << arr[i] << " ";
49          }
50          cout << endl;
51      }
52
53
54  };
55
56  int main() {
57      self_Adjusting_Arr list;
58      list.insert_front(10);
59      list.insert_front(20);
60      list.insert_front(30);
61      list.insert_front(40);
62
63      cout<<"initial list:";
64      list.display();
65
66      list.find(20);
67      cout <<"After accessing 20: ";
68      list.display();
69
70      return 0;
71  }
```

```
"C:\Users\user\OneDrive - Ric x + v
initial list:40 30 20 10
After accessing 20: 20 40 30 10

Process returned 0 (0x0) execution time : 0.370 s
Press any key to continue.
```

B

C:\Users\user > OneDrive - Richfield Graduate Institute of Technology > Documents > 2nd year semester 2 > Programming assignment > question_2b.cpp > ...

```

1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* next;
7  };
8
9  class SelfAdjustingList {
10 private:
11     Node* head;
12
13 public:
14     SelfAdjustingList() {
15         head = nullptr;
16     }
17
18     // Insert at the front
19     void insertFront(int value) {
20         Node* newNode = new Node{value, head};
21         head = newNode;
22     }
23
24     // Find and move to front
25     bool find(int value) {
26         if (head == nullptr || head->data == value)
27             return head != nullptr;
28
29         Node* prev = nullptr;
30         Node* curr = head;
31
32         // Search for the node
33         while (curr != nullptr && curr->data != value) {
34             prev = curr;
35             curr = curr->next;
36         }
37
38         // Find and move to front
39         bool find(int value) {
40             if (head == nullptr || head->data == value)
41                 return head != nullptr;
42
43             Node* prev = nullptr;
44             Node* curr = head;
45
46             // Search for the node
47             while (curr != nullptr && curr->data != value) {
48                 prev = curr;
49                 curr = curr->next;
50             }
51
52             // If not found
53             if (curr == nullptr)
54                 return false;
55
56             // Move the found node to the front
57             prev->next = curr->next;
58             curr->next = head;
59             head = curr;
60             return true;
61         }
62
63     // Display the list
64     void display() {
65         Node* temp = head;
66         while (temp != nullptr) {
67             cout << temp->data << " ";
68             temp = temp->next;
69         }
70         cout << endl;
71     }
72 };

```

C:\Users\user > user > OneDrive - Richfield Graduate Institute of Technology > Documents > 2nd year semester 2 > Programming assignment > question_2b.cpp > main()

```
9  class SelfAdjustingList {
49      // Display the list
50  void display() {
51      Node* temp = head;
52      while (temp != nullptr) {
53          cout << temp->data << " ";
54          temp = temp->next;
55      }
56      cout << endl;
57  }
58  };
59
60  int main() {
61      SelfAdjustingList list;
62      list.insertFront(10);
63      list.insertFront(20);
64      list.insertFront(30);
65      list.insertFront(40);
66
67      cout << "Initial list: ";
68      list.display();
69
70      list.find(10);
71      cout << "After accessing 10: ";
72      list.display();
73
74      return 0;
75  }
```

```
"C:\Users\user\OneDrive - Ric  X + v
Initial list: 40 30 20 10
After accessing 10: 10 40 30 20

Process returned 0 (0x0)   execution time : 0.214 s
Press any key to continue.
```

C

If all elements, I have a constant access probability p_i such that: $\sum_{i=1}^n p_i = 1$.

With Move-To-Front heuristic applied, more frequent elements will be closer to the front because each access moves the element to the front. With a lot of accesses, an equilibrium is reached so that the expected position of the element I is approximately proportional to its access probability (Sleator & Tarjan, 1985).

The expected position $E[pos(i)]$ satisfies:

$$E[pos(i)] \approx \frac{1}{p_i}$$

So elements with large p_i then have small, expected position, and hence stay near the front of the list.

This minimizes average search time most sought-after items are found earlier, and therefore makes the self-adjusting list practical in use. (Cormen, et al., 2009) (Sleator & Tarjan, 1985)

Question 3

A

After we have a threaded binary tree, the nullptr pointers of each node (right or left) are replaced with threads pointing towards the inorder predecessor or inorder successor of the node. But for distinction of whether a pointer is really a child or a thread, we use Boolean flags in all nodes:

```
Question_3.cpp > ...
1  struct Node {
2      int data;
3      Node* left;
4      Node* right;
5      bool leftThread; // true if left points to inorder predecessor
6      bool rightThread; // true if right points to inorder successor
7  };
8
```

This allows efficient traversal and insertion without recursion or stack usage (Knuth, 1998) (Cormen, et al., 2009).

B

```
Question_3.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* left;
7      Node* right;
8      bool leftThread;
9      bool rightThread;
10 };
11
12 class ThreadedBST {
13 private:
14     Node* root;
15
16 public:
17     ThreadedBST() {
18         root = nullptr;
19     }
20
21     // Helper function to create a new node
22     Node* createNode(int key) {
23         Node* node = new Node;
24         node->data = key;
25         node->left = node->right = nullptr;
26         node->leftThread = node->rightThread = true;
27         return node;
28     }
29
30     // Insert a node into the threaded BST
31     void insert(int key) {
32         if (root == nullptr) {
33             root = createNode(key);
34             return;
35         }
36
37         Node* curr = root;
```

```

Question_3.cpp > ...
12  class ThreadedBST {
31  void insert(int key) {
38      Node* parent = nullptr;
39
40      while (curr != nullptr) {
41          if (key == curr->data) {
42              cout << "Duplicate keys not allowed\n";
43              return;
44          }
45
46          parent = curr;
47          if (key < curr->data) {
48              if (!curr->leftThread)
49                  curr = curr->left;
50              else
51                  break;
52          } else {
53              if (!curr->rightThread)
54                  curr = curr->right;
55              else
56                  break;
57          }
58      }
59
60      Node* node = createNode(key);
61
62      if (key < parent->data) {
63          node->left = parent->left;
64          node->right = parent;
65          parent->leftThread = false;
66          parent->left = node;
67      } else {
68          node->left = parent;
69          node->right = parent->right;
70          parent->rightThread = false;
71          parent->right = node;
72      }

```

```

Question_3.cpp > ThreadedBST > insert(int)
12  class ThreadedBST {
31  void insert(int key) {
67      } else {
73  }
74
75  // Inorder traversal without recursion or stack
76  void inorder() {
77      Node* curr = leftMost(root);
78      while (curr != nullptr) {
79          cout << curr->data << " ";
80          curr = inorderSuccessor(curr);
81      }
82      cout << endl;
83  }
84
85  // Find the leftmost node
86  Node* leftMost(Node* node) {
87      if (node == nullptr) return nullptr;
88      while (!node->leftThread)
89          node = node->left;
90      return node;
91  }
92
93  // Find the inorder successor
94  Node* inorderSuccessor(Node* node) {
95      if (node->rightThread)
96          return node->right;
97      node = node->right;
98      while (!node->leftThread)
99          node = node->left;
100     return node;
101 }
102
103 // Delete a node (basic case handling)
104 void deleteNode(int key) {
105     Node* curr = root;
106     Node* parent = nullptr;

```

Question_3.cpp > ThreadedBST > insert(int)

```
12 class ThreadedBST {
104 void deleteNode(int key) {
106     Node* parent = nullptr;
107     bool found = false;
108
109     // Search for the node
110     while (curr != nullptr) {
111         if (key == curr->data) {
112             found = true;
113             break;
114         }
115         parent = curr;
116         if (key < curr->data) {
117             if (!curr->leftThread)
118                 curr = curr->left;
119             else
120                 break;
121         } else {
122             if (!curr->rightThread)
123                 curr = curr->right;
124             else
125                 break;
126         }
127     }
128
129     if (!found) {
130         cout << "Key not found.\n";
131         return;
132     }
133
134     // For simplicity, handle leaf node deletion only
135     if (curr->leftThread && curr->rightThread) {
136         if (parent == nullptr)
137             root = nullptr;
138         else if (curr == parent->left) {
139             parent->leftThread = true;
140             parent->left = curr->left;
141         }
142     }
143     delete curr;
144     cout << "Node deleted.\n";
145 } else {
146     cout << "Deletion of non-leaf nodes not shown here for brevity.\n";
147 }
148 }
149
150 };
151
152
153 int main() {
154     ThreadedBST tree;
155     tree.insert(50);
156     tree.insert(30);
157     tree.insert(70);
158     tree.insert(20);
159     tree.insert(40);
160     tree.insert(60);
161     tree.insert(80);
162
163     cout << "Inorder traversal of threaded BST:\n";
164     tree.inorder();
165
166     tree.deleteNode(20);
167     cout << "After deleting 20:\n";
168     tree.inorder();
169
170     return 0;
171 }
172
```

Question_3.cpp > ThreadedBST

```
12 class ThreadedBST {
104 void deleteNode(int key) {
135     if (curr->leftThread && curr->rightThread) {
138         else if (curr == parent->left) {
140             parent->left = curr->left;
141         } else {
142             parent->rightThread = true;
143             parent->right = curr->right;
144         }
145         delete curr;
146         cout << "Node deleted.\n";
147     } else {
148         cout << "Deletion of non-leaf nodes not shown here for brevity.\n";
149     }
150 }
151
152
153 int main() {
154     ThreadedBST tree;
155     tree.insert(50);
156     tree.insert(30);
157     tree.insert(70);
158     tree.insert(20);
159     tree.insert(40);
160     tree.insert(60);
161     tree.insert(80);
162
163     cout << "Inorder traversal of threaded BST:\n";
164     tree.inorder();
165
166     tree.deleteNode(20);
167     cout << "After deleting 20:\n";
168     tree.inorder();
169
170     return 0;
171 }
172
```

```
"C:\Users\user\OneDrive - Ric  X + v
Inorder traversal of threaded BST:
20 30 40 50 60 70 80
Node deleted.
After deleting 20:
30 40 50 60 70 80

Process returned 0 (0x0)   execution time : 0.210 s
Press any key to continue.
```

This implementation minimizes wasted pointer space and allows constant-space traversal (Weiss, 2014)

C

The advantage of threaded binary trees is that it is simple to perform in-order traversal without recursion or additional stack space. A standard binary search tree contains numerous nullptr pointers, which consume space. Threaded trees reuse null pointers by linking them with a node's predecessor or successor in in-order, which not only saves space but also improves traversal algorithms to be simpler and faster (Knuth, 1998). Along these directions, it is possible to traverse the tree sequentially in order with constant auxiliary space, improving overall performance for processes where there is a requirement for regular inorder traversal. (Cormen, et al., 2009). Also threaded trees simplify searching for the successor and predecessor because the threads provide direct links to the next or previous nodes, reducing computational overhead (Weiss, 2014). This makes threaded trees particularly advantageous in systems with limited memory or where efficient ordered data access is a priority.

Question 4

A

```
question_4.cpp > main()
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  int increasefine(int day){
6      // 10^(2^(day-1))
7
8      unsigned long long fine = pow(10,pow(2,day-1));
9
10     return fine;
11 }
12
13
14 int main() {
15     cout << "Enter the number of days overdue: ";
16     int day;
17
18     cin >> day;
19     int fine = increasefine(day);
20     cout << "The fine for being late is: "<<fine <<endl;
21     return 0;
22 }
23
24
25
```

```
"C:\Users\user\OneDrive - Ric  X + v
Enter the number of days overdue:
```

```
"C:\Users\user\OneDrive - Ric  X + v
Enter the number of days overdue: 4
The fine for being late is: 99999999

Process returned 0 (0x0)   execution time : 3.384 s
Press any key to continue.
```

This program computes the fine using the formula $F(N) = 10^{(2^{N-1})}$

). The pow() function of <cmath> performs the exponentials with efficiency (Deitel & Deitel, 2021). When N is large, the fine grows too quickly to be processed by the standard data types, and the computed result can overflow (Weiss, 2014).

B

To find the smallest integer N such that $F(N) \geq D$: $10^{(2^{N-1})} \geq D$.

Take base-10 logs: $2^{N-1} \geq \log_{10} D$.

Take base-2 logs: $N - 1 \geq \log_2(\log_{10} D)$,

So the smallest integer solution is:

$$N = \lceil 1 + \log_2(\log_{10} D) \rceil \text{ so this equals to } N = O(\log \log D)$$

This means that the fine grows so rapidly that it only takes a double logarithmic number of days to reach enormous amounts (Cormen, et al., 2022) , because the exponent doubles each day, the growth is tower-like, and even small increases in N cause huge increases in $F(N)$ (Aho, et al., 1983)

References

- Aho, A. V., Hopcroft, J. E. & Ullman, J. D., 1983. *Data Structures and Algorithms*.. s.l.:Reading, MA: Addison-Wesley..
- Anon., 1998. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*.. 3 ed. s.l.:Reading, MA: Addison-Wesley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C., 2009. *Introduction to Algorithms*.. 3 ed. Cambridge, MA: MIT Press..
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C., 2022. *Introduction to Algorithms*.. 4 ed. Cambridge, MA: MIT Press.
- CP-Algorithms, 2024. *Josephus Problem*. Available at:. [Online]
Available at: https://cp-algorithms.com/others/josephus_problem.html
[Accessed 22 october 2025].
- Deitel, P. & Deitel, H., 2021. *C++ How to Program*.. 11 ed. Harlow: Pearson Education.
- Knuth, D., 1998. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3 ed. s.l.:Reading, MA: Addison-Wesley..
- Sleator, D. D. & Tarjan, R. E., 1985. Self-adjusting binary search trees. *Journal of the ACM*, p. 652–686.
- Weiss, M., 2014. *Data Structures and Algorithm Analysis in C++*. 4 ed. Boston: Pearson..